

ia-26/PS0.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import functions as f
5
6 class Particle:
7
8     def __init__(self, dim, bounds, objective_function):
9
10         self.objective_function = objective_function
11
12         self.position = np.random.uniform(
13             bounds[0],
14             bounds[1],
15             dim
16         )
17
18         self.velocity = np.random.uniform(
19             -1,
20             1,
21             dim
22         )
23
24         self.best_position = self.position.copy()
25
26         self.best_value = self.objective_function(
27             self.position
28         )
29
30     def update_velocity(
31         self,
32         global_best,
33         w,
34         c1,
35         c2
36     ):
37
38         r1 = np.random.rand(len(self.position))
39         r2 = np.random.rand(len(self.position))
40
41         cognitive = (
42             c1 * r1 *
43             (self.best_position - self.position)
44         )
45
46         social = (
47             c2 * r2 *
48             (global_best - self.position)
49         )
50

```

```

51         self.velocity = (
52             w * self.velocity
53             + cognitive
54             + social
55         )
56
57     def update_position(self, bounds):
58
59         self.position += self.velocity
60
61         self.position = np.clip(
62             self.position,
63             bounds[0],
64             bounds[1]
65         )
66
67         value = self.objective_function(
68             self.position
69         )
70
71         if value < self.best_value:
72
73             self.best_value = value
74
75             self.best_position = (
76                 self.position.copy()
77             )
78
79     def particle_swarm_optimization(
80         objective_function,
81         dim=2,
82         n_particles=50,
83         max_iter=200,
84         bounds=(-100, 100),
85         w=0.7,
86         c1=1.5,
87         c2=1.5
88     ):
89
90         swarm = [
91             Particle(
92                 dim,
93                 bounds,
94                 objective_function
95             )
96             for _ in range(n_particles)
97         ]
98
99         global_best_position = (
100             swarm[0].best_position.copy()
101         )
102
103         global_best_value = (

```

```

104     swarm[0].best_value
105 )
106
107 history = []
108
109 for particle in swarm:
110
111     if particle.best_value < global_best_value:
112
113         global_best_value = (
114             particle.best_value
115         )
116
117         global_best_position = (
118             particle.best_position.copy()
119         )
120
121 for iteration in range(max_iter):
122
123     for particle in swarm:
124
125         particle.update_velocity(
126             global_best_position,
127             w,
128             c1,
129             c2
130         )
131
132         particle.update_position(bounds)
133
134         if particle.best_value < global_best_value:
135
136             global_best_value = (
137                 particle.best_value
138             )
139
140             global_best_position = (
141                 particle.best_position.copy()
142             )
143
144             print(
145                 f"Iteração {iteration+1:5d}"
146                 f" | Melhor valor: "
147                 f"{global_best_value:.10f}"
148             )
149
150         history.append(global_best_value)
151
152
153 return (
154     global_best_position,
155     global_best_value,
156     history

```

```

157 )
158
159 def plot_convergence(history):
160
161     plt.figure(figsize=(8, 5))
162
163     plt.plot(history)
164
165     plt.title("Convergência do PSO")
166
167     plt.xlabel("Iteração")
168     plt.ylabel("Melhor valor")
169
170     plt.yscale("log")
171
172     plt.grid(True)
173
174     plt.show()
175
176
177 if __name__ == "__main__":
178
179     objective_function = f.f1
180
181     bounds = (-100, 100)
182
183     best_position, best_value, history = (
184         particle_swarm_optimization(
185             objective_function=objective_function,
186             dim=30,
187             n_particles=100,
188             max_iter=1000,
189             bounds=bounds
190         )
191     )
192
193     plot_convergence(history)
194
195     print("\n===== RESULTADO FINAL =====")
196
197     print("Melhor posição:")
198     print(best_position)
199
200     print("\nMelhor valor:")
201     print(best_value)

```